

La interoperabilidad entre los sistemas

M6: Desarrollo de aplicaciones jee con spring framework

|AE5: Utilizar spring framework para la disponibilización de un servicio rest que da solución a un problema de interoperabilidad.

Introducción

El desarrollo de aplicaciones empresariales ha evolucionado enormemente en los últimos años gracias a frameworks como Spring, que facilitan la construcción de sistemas escalables, flexibles y mantenibles. En esta lección **veremos los conceptos básicos para construir una aplicación JEE con Spring Framework, centrándonos en la interoperabilidad entre sistemas.**

Abordaremos temas como la arquitectura REST, el formato JSON para intercambio de datos, el consumo de servicios REST con RestTemplate y la creación de nuestra propia API REST con Spring MVC. También veremos buenas prácticas en el diseño de APIs RESTful y cómo asegurarlas con JWT.

Al finalizar, tendrás las bases necesarias para comenzar a desarrollar aplicaciones JEE que se integren fácilmente con otros sistemas mediante servicios web REST, aprovechando toda la potencia de Spring Framework.

Aprendizaje esperado

Al finalizar esta lección podrás:

- Utilizar Spring Framework para la disponibilización de un servicio REST que da solución a un problema de interoperabilidad.

La interoperabilidad entre los sistemas

La interoperabilidad es la **capacidad de dos o más sistemas o componentes para intercambiar información y utilizar la información intercambiada.**

Cuando hablamos de sistemas, la interoperabilidad se refiere a la habilidad de interactuar entre dos sistemas de tecnología de información, como por ejemplo una aplicación web y una base de datos.

Para lograr la interoperabilidad entre sistemas es necesario que:

- **Hablen un lenguaje común:** esto se logra definiendo protocolos y formatos estándar para la comunicación.
- **Tengan interfaces compatibles:** las APIs de cada sistema deben poder conectarse e interactuar sin problemas.

Protocolos de intercambio de datos

Un protocolo de intercambio de datos es un **conjunto de reglas y estándares que permiten que dos sistemas se comuniquen entre sí para intercambiar información.**

Los protocolos de intercambio de datos cumplen varias funciones importantes:

- Definen un formato estándar para representar los datos, como XML, JSON, CSV, etc. Esto permite que sistemas distintos entiendan la información.
- Establecen cómo debe realizarse la comunicación: solicitud, respuesta, secuenciación, etc. Por ejemplo, el protocolo HTTP define verbos como GET, POST, PUT.
- Permiten la detección y corrección de errores. Los protocolos pueden especificar códigos de error y solicitudes de retransmisión.
- Proporcionan seguridad, como encriptación y autenticación. Por ejemplo, HTTPS es HTTP sobre SSL.
- Facilitan la integración al actuar como un "lenguaje común". Los sistemas solo necesitan "hablar" ese protocolo para entenderse.

Algunos protocolos comunes para el intercambio de datos entre sistemas:

SOAP: Simple Object Access Protocol, permite el intercambio de mensajes XML.

REST: Transferencia de Estado Representacional, utiliza HTTP para las operaciones CRUD.

JSON: Notación de Objetos JavaScript, formato ligero para intercambio de datos.

XML: Lenguaje de Marcado Extensible, permite definir formatos personalizados.

Elegir el protocolo adecuado es clave para permitir la comunicación entre sistemas heterogéneos.

Estado Representacional de Transferencia (REST)

REST (**RE**presentational **State** Transfer) es un **estilo de arquitectura software** para sistemas hipermedia distribuidos como la World Wide Web. Fue propuesto por Roy Fielding en el año 2000.

REST define un conjunto de principios y restricciones que buscan **aprovechar las ventajas de los protocolos y estándares** del mundo web, como HTTP, URI y formatos como XML y JSON.

Las principales características de los sistemas RESTful son:

- Interacciones cliente-servidor sin estado: cada petición contiene toda la información necesaria.
- **Operaciones bien definidas** (GET, POST, PUT, DELETE) con significado estandarizado.
- **Sintaxis universal** para identificar recursos (URI). Por ejemplo, <https://ejemplo.com/usuarios>
- **Representaciones estándar** (XML, JSON, etc) intercambiadas en las interacciones.
- **Comunicación** mediante HTTP sin estado.

- REST es el estilo arquitectónico más utilizado actualmente para APIs web y móviles. Facilita la integración al usar estándares abiertos en lugar de tecnologías patentadas.

JSON: Notación para traspaso de información

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos. Permite **representar de forma sencilla estructuras de datos y objetos como cadenas de texto**.

Características principales:

- Sintaxis derivada de la notación de objetos JavaScript pero completamente independiente del lenguaje.
- Basado en parejas nombre/valor y listas ordenadas. Permite expresar objetos anidados.
- De fácil lectura para humanos y máquinas. Más compacto que XML.
- Independiente del lenguaje pero con parsers disponibles en la mayoría de lenguajes.
- Se puede leer y generar fácilmente mediante JavaScript.

JSON es el formato ideal para el intercambio de datos en APIs web modernas y servicios REST:

- Es mucho más ligero y rápido que XML.
- Es nativo de JavaScript por lo que se integra perfectamente en aplicaciones web.
- Es compatible con la mayoría de lenguajes y plataformas.

Por estas razones, JSON se ha convertido en el estándar de facto para intercambiar datos en servicios web, desplazando a formatos como XML.

Consumiendo un servicio REST con Spring y RestTemplate

RestTemplate es una clase de Spring que nos permite consumir servicios REST y realizar solicitudes HTTP de forma sencilla. Nos abstrae de trabajar directamente con la API HTTP de Java.

Los principales métodos que provee **RestTemplate** en Spring para consumir servicios REST son:

getForObject(): Envía una petición GET y recibe el cuerpo de la respuesta directamente mapeado a una instancia de una clase.

getForEntity(): Similar a **getForObject** pero devuelve la respuesta completa como **ResponseEntity**.

postForObject(): Envía una petición POST con un objeto en el cuerpo y recibe el objeto resultado mapeado a una clase.

postForEntity(): Similar a **postForObject** pero devuelve una **ResponseEntity**.

put(): Envía una petición PUT con un objeto en el cuerpo.

delete(): Envía una petición DELETE.

exchange(): Permite construir solicitudes más complejas y personalizadas.

Estos métodos permiten de forma sencilla:

- Realizar operaciones con los principales verbos HTTP (GET, POST, PUT, DELETE).
- Enviar objetos Java en el cuerpo y recibir objetos mapeados desde la respuesta.
- Obtener metadatos de la respuesta en **ResponseEntity**.
- Intercambiar datos en formatos como JSON.
- Manejar errores y excepciones de forma transparente.

Para utilizar **RestTemplate** necesitamos agregar la dependencia **spring-web** en nuestro proyecto:

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-web</artifactId>
  <version>5.3.20</version>
</dependency>
```

Y luego inyectamos RestTemplate en nuestra clase. El RestTemplate típicamente se inyecta y utiliza en los controllers de Spring MVC.

```
@RestController
@RequestMapping("/api")
public class MyController {

    @Autowired
    RestTemplate restTemplate;

    @GetMapping("/users")
```

De esta forma, encapsulamos las llamadas a servicios externos en los controllers, manteniendo la lógica del negocio separada. El RestTemplate nos resulta muy útil en aplicaciones que consumen múltiples APIs.

Con `getForObject` enviamos una petición GET y recibimos el cuerpo de la respuesta directamente mapeado a una instancia de la clase User.

En el controlador agregaremos el objeto modelo y la vista a devolver en un **ModelAndView**. Luego, en la vista **users.jsp** accedemos al modelo:

```
@RestController
@RequestMapping("/api")
public class MyController {
    @Autowired
    RestTemplate restTemplate;

    @GetMapping("/users")
    public ModelAndView getUsers() {

        // Llamada a API externa
        List<User> users = restTemplate.getForObject("https://api.example.com/users", List.class);

        // Agregar al modelo
        ModelAndView model = new ModelAndView("users");
        model.addObject("users", users);

        return model;
    }
}
```

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
<head>
  <title>Users</title>
</head>
<body>
  <h1>Users</h1>

  <ul>
    <c:forEach items="${users}" var="user">
      <li>${user.name} - ${user.email}</li>
    </c:forEach>
  </ul>

</body>
</html>
```

Principios de diseño de una API de REST

Los principios para desarrollar APIs REST se refieren a un conjunto de **directrices y buenas prácticas que se deben seguir al diseñar e implementar una API** que cumpla con el estilo arquitectónico REST.

Estos principios sirven para:

- Crear APIs estandarizadas, simples y predecibles. Al seguir los mismos principios, diferentes APIs tendrán similitudes y será más fácil aprender a consumirlas.
- Aprovechar mejor las características de HTTP y los estándares web. Por ejemplo, usando correctamente los verbos y códigos de estado HTTP.
- Crear interfaces flexibles que permitan la evolución y extensibilidad en el futuro.
- Minimizar acoplamiento entre cliente y servidor. Al ser stateless y cacheable se reduce la dependencia.

- Permitir que distintos clientes puedan consumir la API fácilmente.
- Desacoplar la API del backend, por ejemplo separando en capas.
- Crear APIs que puedan ser escaladas horizontalmente si se requiere.

Algunos de los principales principios de diseño son:

- **Recursos identificados mediante URIs:** Cada recurso (usuarios, posts, etc) debe tener una URI única. Por ejemplo, /api/users.
- **Verbos HTTP para las operaciones:** GET para leer recursos, POST para crear, PUT para modificar, DELETE para eliminar.
- **Respuestas autogeneradas:** La API devuelve la representación canónica del recurso, no códigos de estado.
- **Sin estado:** Las peticiones HTTP deben contener toda la información necesaria, no deben depender de estado previo.
- **Capas:** Separar lógica de la API de la de negocio. La API solo maneja URIs, verbos, entradas/salidas.
- **Cacheable:** Las respuestas deben declarar si son cacheables mediante headers como Cache-Control.
- **Interfaz uniforme:** La interfaz entre cliente y servidor debe ser consistente y estandarizada.
- **HATEOAS:** Hypermedia as the Engine of Application State. La API debe devolver links a recursos relacionados.
- **Documentación:** Documentar claramente las URIs, parámetros, respuestas, códigos de error, etc.
- Siguiendo estos principios se pueden desarrollar APIs RESTful escalables, flexibles y con una fuerte cohesión.

Creando una API REST con Spring MVC

Spring MVC facilita la creación de APIs RESTful proveyendo anotaciones como `@RestController` o `@RequestMapping` para mapear requests HTTP a métodos de controladores.

Se define un controlador por recurso, por ejemplo `UserController` para `/users`. Y se implementan los métodos CRUD invocando a los repositorios.

También se mapean solicitudes GET, POST, PUT, DELETE a los métodos del controlador con `@GetMapping`, `@PutMapping`, etc.

Se inyectan los repositorios de datos para acceder a la capa de persistencia y Spring convierte automáticamente objetos Java a JSON para las respuestas.

Además, se pueden agregar facilidades como validación, manejo de errores, paginación, caching, documentación, etc.

Para una API completa se crean los modelos, controladores y vistas JSP/Thymeleaf.

La API puede desplegarse en un servidor de aplicaciones como Tomcat o directamente como un `jar` con el starter de Spring Boot.

Veamos algunos pasos esenciales:

1. Crear el proyecto Spring Boot e importar dependencias:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

2. Crear el modelo y repositorio de datos. Por ejemplo, una entidad Usuario y un repositorio JPA para acceder a la base de datos.
3. Crear el controlador REST: Para esto añadimos la anotación `@RestController`.

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    @Autowired
    private UserRepository userRepo;

    // GET, POST, PUT, DELETE methods...

}
```

4. Mapear URLs a métodos del controlador:

```
@GetMapping("/")
List<User> getUsers() {
    return userRepo.findAll();
}

@PostMapping("/")
User createUser(@RequestBody User user) {
    return userRepo.save(user);
}
```

También podríamos hacer una llamada a una api externa y devolver una vista con la información a través del modelo:

```
@GetMapping("/external")
public ModelAndView getExternalBooks() {

    String url = "https://api.example.com/books";
    List<Book> books = restTemplate.getForObject(url, List.class);

    ModelAndView mav = new ModelAndView("external");
    mav.addObject("books", books);

    return mav;
}
```

Securización de una API REST mediante JWT

Una de las formas más utilizadas actualmente para securizar APIs REST es mediante la implementación de JSON Web Tokens (**JWT**). Los JWT **permiten la autenticación stateless entre el cliente y el servidor**, ya que encapsulan la información de acceso en un token firmado que se envía en cada request.

Spring Framework provee el soporte necesario para incorporar JWT y seguridad basada en tokens dentro de una API REST creada con Spring MVC. Mediante la integración con Spring Security es posible agregar filtros de autenticación JWT, endpoints protegidos, roles y autorizaciones.

Un JWT contiene tres partes:

- **Header:** algoritmo usado (HS256, RS256, etc) y tipo de token.
- **Payload:** claims o afirmaciones sobre la entidad. Puede contener datos como usuario, roles, etc.
- **Firma:** permite verificar la integridad del mensaje.

El JWT se codifica en Base64 y se envía en el header Authorization de cada request.

El servidor valida la firma para autenticar al cliente. Así se implementa la autenticación stateless.

Ejemplo en una API REST

1. El cliente se autentica y obtiene un JWT
2. En cada request envía el JWT:

<!--> Authorization: Bearer <jwt>

3. El servidor valida el JWT:
 - Verifica la firma con la clave secreta
 - Extrae claims como username, roles, etc
 - Crea el contexto de seguridad
4. Permite o deniega el acceso a los recursos

Implementación en Spring con Spring Security y la librería JWT

1. Endpoint para generar el JWT:
2. Filtro JwtAuthenticationFilter para validar y parsear el JWT
3. Anotaciones como @PreAuthorize para restringir acceso

Cierre

A lo largo de esta lección hemos visto cómo implementar APIs RESTful de forma rápida y sencilla utilizando el potente framework Spring.

Comenzamos entendiendo conceptos cruciales como la interoperabilidad entre sistemas, el estilo arquitectónico REST y el uso de JSON para intercambio de datos. Luego, de la mano de Spring, pudimos desarrollar paso a paso una API exponiendo recursos, implementando lógica de negocio y bases de datos, hasta finalmente agregar seguridad con JWT.

Spring MVC, REST y Spring Boot nos permitieron construir con buenas prácticas una API REST escalable y empresarial, lista para ser consumida por múltiples clientes. Gracias a los conocimientos adquiridos, ahora contamos con las herramientas necesarias para desarrollar e integrar soluciones web modernas en Java.

¡Nos vemos de nuevo, más adelante! 

Referencias

- Universidad de Alicante. (s.f.). Interoperabilidad. Servicio de Interoperabilidad, Universidad de Alicante. Recuperado de <https://si.ua.es/es/interoperabilidad/interoperabilidad.html>
- IBM Corporation. (s.f.). What is system interoperability? IBM Developer. Recuperado de <https://developer.ibm.com/articles/what-is-system-interoperability/mediacenter.ibm.com+2community.ibm.com+2community.ibm.com+2>
- Oracle Corporation. (s.f.). Oracle documentation – interoperability and related APIs [Documentación técnica]. Oracle. Recuperado de [Oracle | Protocolos de red para programadores.](#)
- Red Hat, Inc. (s.f.). What is REST? Red Hat. Recuperado de <https://www.redhat.com/es/topics/api/what-is-rest>
- Oracle Corporation. (s.f.). Overview of REST in Oracle Cloud 18c [Documentación técnica]. Oracle. Recuperado de https://docs.oracle.com/cd/E52199_01/Cloud.18C/DevelopingApplications/content/rest_overview-1.html
- The Spring Framework Project. (s.f.). RestTemplate (class in org.springframework.web.client) [JavaDoc API]. Spring Framework. Recuperado de <https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/client/RestTemplate.html>
- Baeldung. (s.f.). A Guide to the RestTemplate. Baeldung. Recuperado de <https://www.baeldung.com/rest-template>
- The Spring Team. (s.f.). Consuming a RESTful Web Service [Getting Started guide]. Spring. Recuperado de <https://spring.io/guides/gs/consuming-rest>
- Red Hat, Inc. (s.f.). Designing RESTful APIs [API Developer guide]. Red Hat. Recuperado de https://docs.redhat.com/es/documentation/red_hat_3scale/2.4/html/api_dev_guide/design-restful-api

- Zalando. (s. f.). *Developing RESTful APIs: a comprehensive set of guidelines* [Online resource]. Zalando Open Source. Recuperado de <https://opensource.zalando.com/restful-api-guidelines/>

¡Muchas gracias!

Nos vemos en la próxima lección

